

Agenda

- ① Introduction to Arrays
- ② Array declaration rules
- ③ Bound checking
- ④ Sorting
- ⑤ Function call by passing arrays
- ⑥ Two dimensional arrays
- ⑦ multi dimensional arrays.

Introduction to Arrays

- Array is a linear collection of similar elements.
- Array is also known as Subscript variable

Write a program to calculate average of 100 numbers.

```
int main()
```

```
{
```

```
    int a[100], i, sum = 0;
```

```
    float avg;
```

```
    printf("Enter 100 nos");
```

```
    for(i=0; i<=99; i++)
```

```
        scanf("%d", &a[i]);
```

```
    for(i=0; i<=99; i++)
```

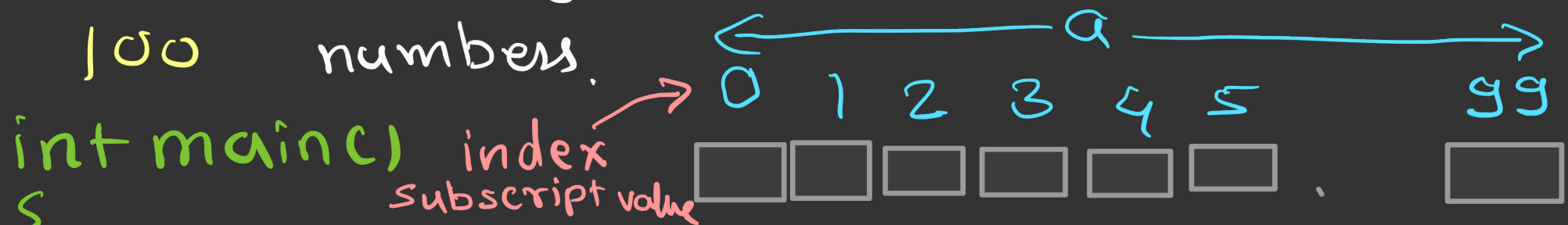
```
        sum = sum + a[i];
```

```
    avg = sum / 100.0;
```

```
    printf("Average is %f", avg);
```

```
    return 0;
```

```
}
```



`a[3] = 30;`

`[]` subscript operator

`a[i]`

`x+y` `3+4`

Array Declaration Rules

① `int a[];` **Error**
↖ Can't be empty

② `int a[5];`
↖ Natural number
• Total number of variables in array
• Not an index

③ `int a[5];`

0	1	2	3	4

local array when not initialized contains garbage values.

whatever is the size of an array it always consumes memory in a sequential fashion.

④ You can initialize array during declaration

```
int a[5] = {10, 50, 30, 70, 20};
```

0	1	2	3	4
10	50	30	70	20

⑤ You cannot initialize an array during declaration more than its size

```
int a[5] = {10, 50, 30, 70, 20, 80, 40};
```

Error

⑥ You can initialize an array during declaration with lesser values than the size of an array.

```
int a[5] = {10, 50};
```

0	1	2	3	4
10	50	0	0	0

Remaining variables in array will contain 0.
and not garbage value.

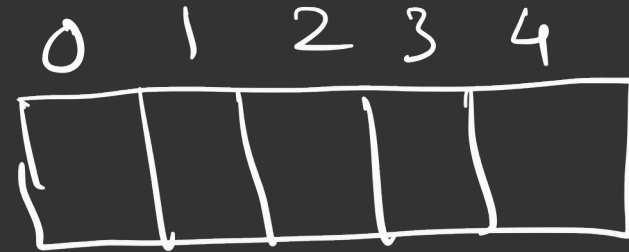
⑦ During declaration you can leave `[]` empty only when you initialize array at the same time.

```
int a[] = {10, 50, 30, 80, 20};
```

Bound Checking

int a[5] = {10, 20, 50, 90, 30, 60, 70}; Error

int i, a[5];



for (i=0; i<=9; i++)

scanf("%d", &a[i]);



Sorting

- Arranging elements in some logical order is known as sorting.
- By default, for numbers sorting means arranging elements in ascending order.

example

given array →

0	1	2	3	4	5	6	7
20	50	90	60	70	80	30	10

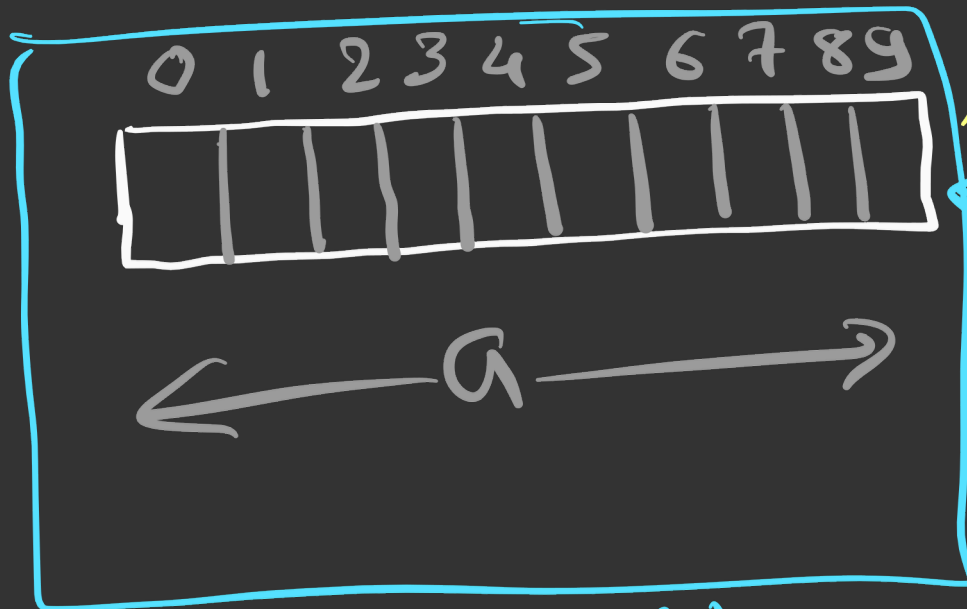
sorted array →

10	20	30	50	60	70	80	90
----	----	----	----	----	----	----	----

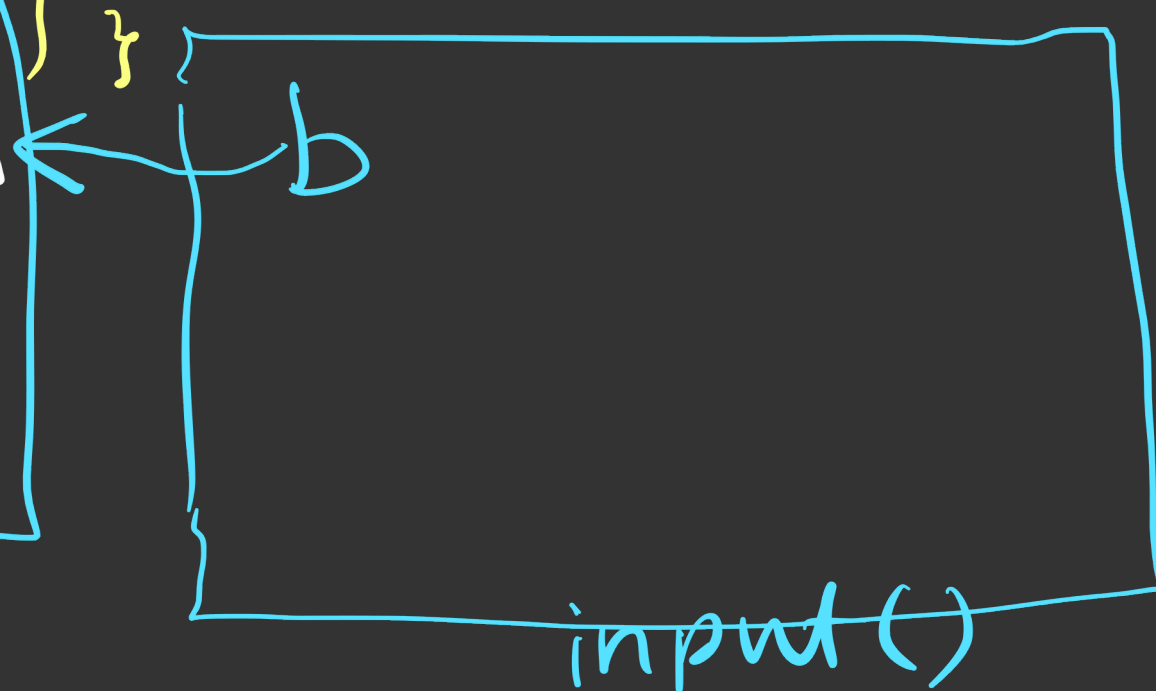
Function call by passing array

```
main()  
{  
    int a[10];  
    input(a);  
}
```

```
void input(int b[])  
{  
    int i;  
    printf("Enter 10 numbers");  
    for(i=0; i<=9; i++)  
        scanf("%d", &b[i]);  
}
```



main()

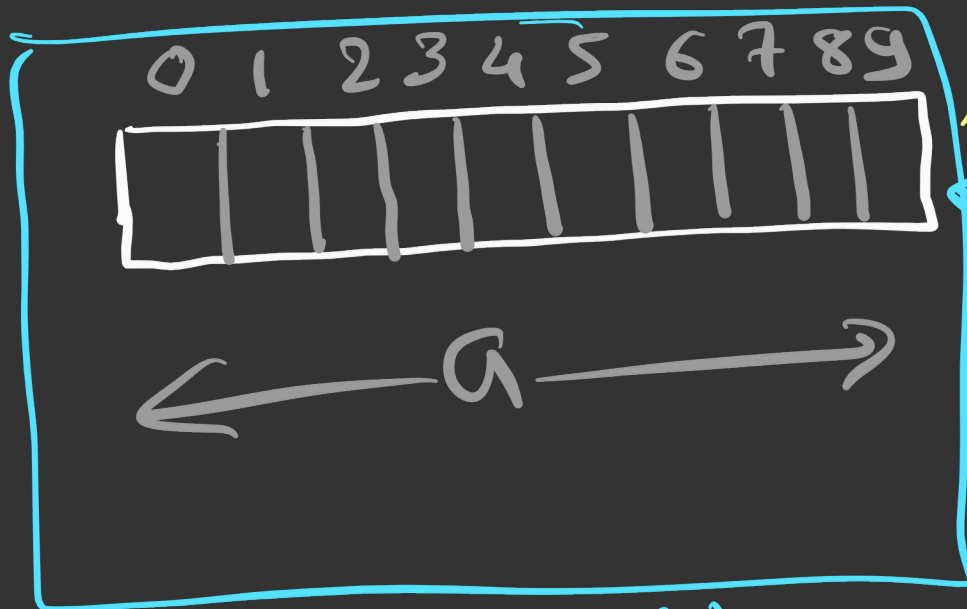


Function call by passing array

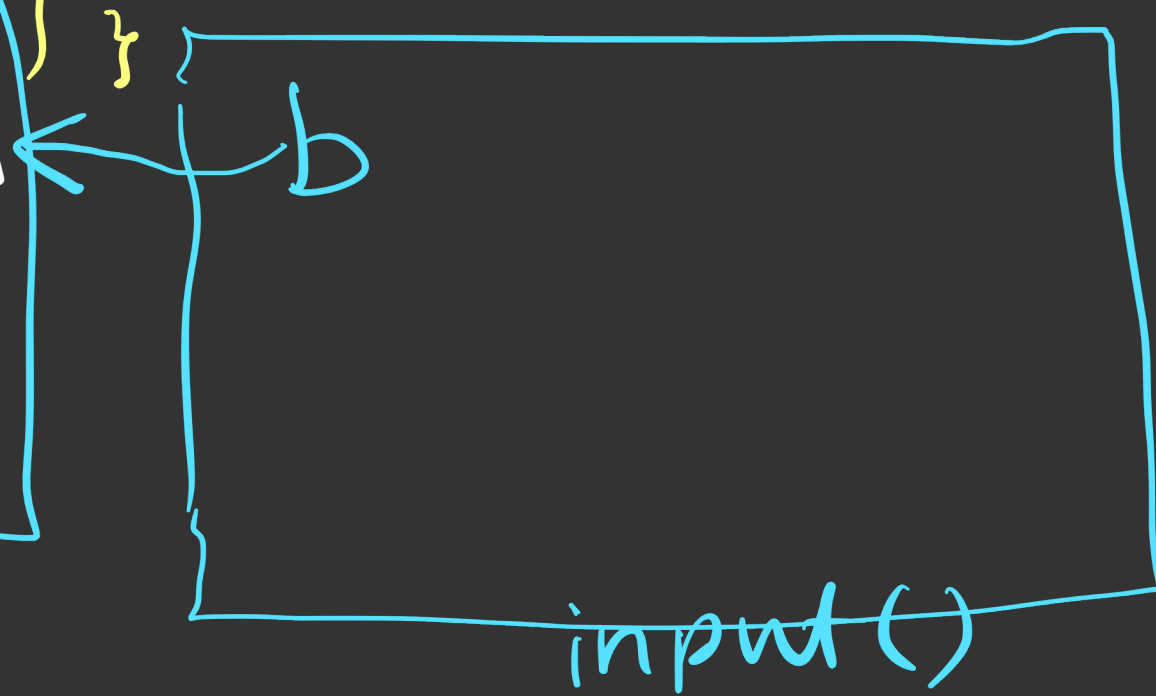
```
main()
{
    int a[5];

    input(a, 5);
}
```

```
void input(int b[], int N)
{
    int i;
    printf("Enter %d numbers", N);
    for (i = 0; i < N; i++)
        scanf("%d", &b[i])
}
```



main()



input()

Two dimensional Arrays

int a[6];

0	1	2	3	4	5

one dimensional Array

int b[2][3];

--	--	--	--	--	--

Two dimensional

int c[4][5][3];

Three dimensional

int a[50]; X

int a[5][10];

0 to 9

↓ ↓ ↓

3 4 5

0 to 4

0 to 9

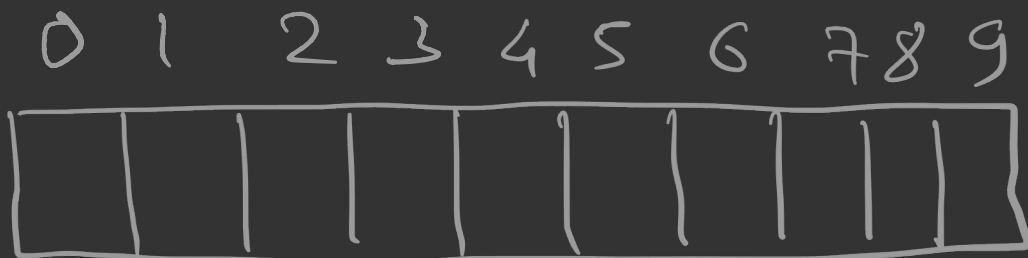
↓ ↓

3 4

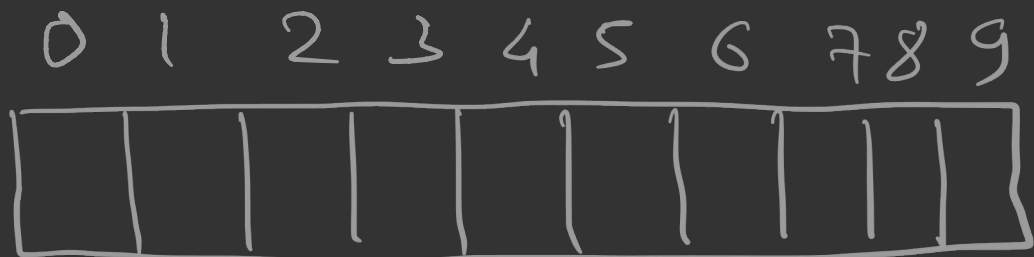
↑



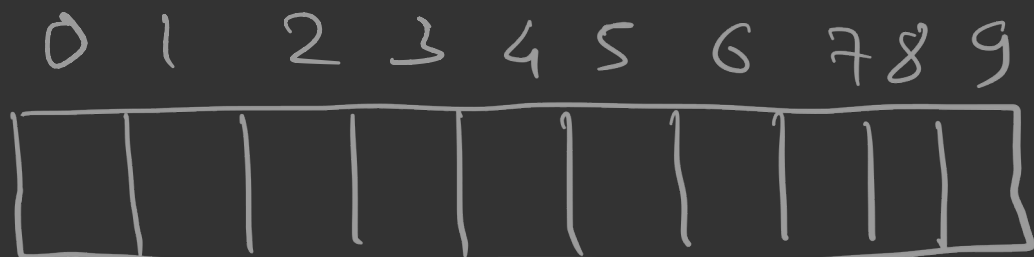
0



1



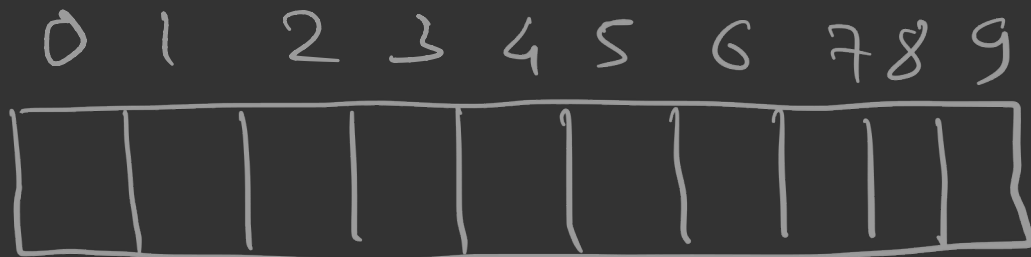
2



3

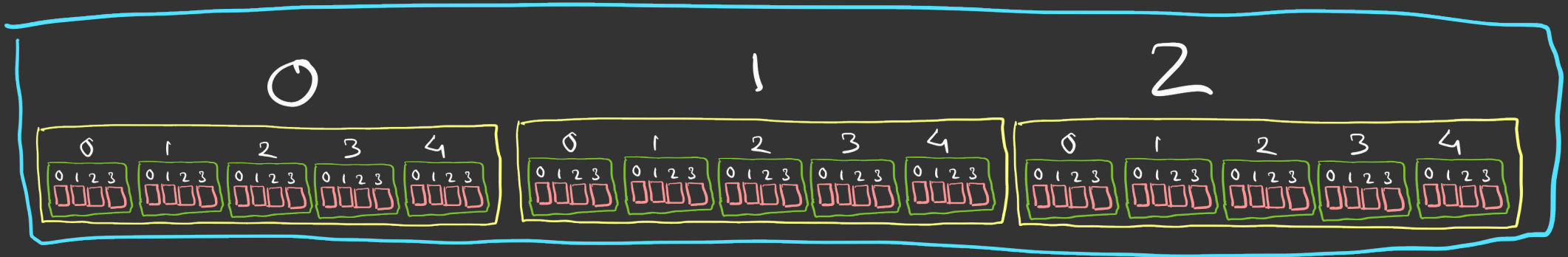


4



Multi-Dimensional Arrays

```
int a[3][5][4];
```



```
a[1][3][2]
```

```
int a[3][4];
```

```
int i, j;
```

```
for(i=0; i<=2; i++)  
{
```

```
    for(j=0; j<=3; j++)
```

```
        scanf("%d", &a[i][j]);
```

```
}
```

i	j
0	0
0	1
0	2
0	3
1	0
1	1
1	2
1	3
2	0
2	1
2	2
2	3

2	0
2	1
2	2
2	3